ЧТО ТАКОЕ FINDING

Подробное объяснение finding-а в проекте EH/Unwind Analyzer: evidence, layer, consequence, confidence, severity, mitigations, correlation и защита перед комиссией.

Главный вопрос	Короткий ответ
Что такое finding?	Структурированный вывод анализатора о конкретном наблюдении, которое важно для reverse engineering. Finding не должен быть голым словом "ошибка". Он должен показывать, что найдено, где найдено, почему это важно и насколько этому выводу можно доверять.
Это verdict?	Нет. Finding - отдельный сигнал. Verdict или case strength строится из набора findings, confidence, severity, mitigations и, если есть, dynamic evidence.
Почему это важно на защите?	Потому что комиссия может спросить: "почему вы так решили?". Хороший finding показывает цепочку рассуждения, а не требует верить инструменту на слово.

1. Базовое определение finding-а

Finding - это не просто строка в отчете и не просто "сработавшая эвристика". В инженерном смысле finding - это оформленное наблюдение анализатора, связанное с конкретным правилом, конкретным местом в объекте анализа и конкретным последствием для интерпретации бинарника.

В контексте проекта EH/Unwind Analyzer finding отвечает на вопрос: "что в PE x64 EH/Unwind metadata выглядит поврежденным, неоднозначным, нестандартным или требующим ручной проверки?" Например: пересечение RUNTIME_FUNCTION диапазонов, невозможность прочитать UNWIND_INFO, handler в неисполняемой секции, writable unwind metadata, слишком сильный reuse одного UnwindInfoRVA.

Важно: finding не обязан сразу означать вредоносность. Он может означать ошибку формата, нестандартный output toolchain-а, артефакт packer-а, runtime/compiler pattern или действительно suspicious вмешательство. Поэтому хороший finding должен отделять наблюдение от интерпретации. Наблюдение должно быть максимально фактическим. Интерпретация должна быть осторожной и проверяемой.

2. Почему короткий finding плох

Плохой finding звучит так: "rf_overlaps detected". Формально это сигнал, но он почти ничего не объясняет. Читатель не видит, какие диапазоны пересеклись, почему это важно для RtlLookupFunctionEntry, насколько надежно это установлено, что может сломаться и что делать дальше.

Хороший finding должен быть длиннее, потому что он выполняет не только роль тревоги, но и роль обучающей доказательной цепочки. Он показывает мышление static-слоя: от PE mapping к .pdata range map, от конкретных адресов к consequence, от consequence к severity, от качества evidence к confidence.

Для твоего проекта это особенно важно, потому что он предназначен для узкой профессиональной темы. Реверсер не должен видеть только "страшное слово". Он должен получить материал для проверки: адреса, секции, связанные записи, affected count, причину важности и следующий ручной шаг.

3. Основные слои finding-а

Слой	Что означает	Пример для EH/Unwind
Evidence	Доказательное наблюдение: конкретные поля, адреса, диапазоны, секции, bytes, counters.	RUNTIME_FUNCTION[12] покрывает RVA 0x1400..0x1480, RUNTIME_FUNCTION[13] покрывает RVA 0x1460..0x1500.
Layer	Уровень механизма, где возникла проблема.	PE mapping, .pdata range map, .xdata decoding, handler metadata, static code-vs-metadata match, dynamic validation.
Rule	Какое правило или ожидание нарушено.	Для одной ControlPc внутри image должна выбираться однозначная RUNTIME_FUNCTION entry.
Expected	Какая нормальная картина ожидалась.	RUNTIME_FUNCTION ranges не пересекаются и отсортированы по BeginAddress.
Observed	Что реально увидел анализатор.	Два диапазона пересекаются на RVA 0x1460..0x1480.
Consequence	Почему это важно, что может сломаться или стать неоднозначным.	RtlLookupFunctionEntry может выбрать не ту metadata или таблица указывает на нестандартную организацию.
Severity	Насколько серьезно наблюдение, если оно реально.	High, потому что нарушается однозначность function range map.
Confidence	Насколько надежно установлено наблюдение или итоговая интерпретация.	High, если оба диапазона прочитаны из корректно mapped Exception Directory.
Mitigation	Что снижает уверенность или смягчает вывод.	Маленький sample, одиночный сигнал, known benign compiler/runtime pattern.

Слой	Что означает	Пример для EH/Unwind
Next step	Что должен проверить реверсер вручную.	Открыть диапазоны в Ghidra, проверить границы функций, сверить с disassembly и dynamic route.

4. Evidence: доказательство, а не мнение

Evidence - это фактическая часть finding-а. Она должна отвечать на вопрос: "на основании каких конкретных данных анализатор сделал вывод?" Для static evidence такими данными являются RVA, file offset, section name, section permissions, значения полей PE/.pdata/.xdata, размер доступных bytes, affected count. Для dynamic evidence - event type, thread id, exception code, first/second chance, handler hit, аргументы RtlAddFunctionTable, trace samples.

Evidence должно быть отделено от объяснения. Например, "handler suspicious" - это не evidence. Evidence - это "handler RVA 0x56010 находится в writable executable section .xw, на него ссылаются 231 функции, dynamic session зафиксировала handler hit". Уже после этого можно формулировать interpretation: "такое сочетание может указывать на centralized exception dispatcher или protection runtime".

На защите это дает сильный ответ: проект не говорит "верьте мне". Он показывает данные, из которых следует вывод. Если комиссия не согласна с интерпретацией, она все равно видит наблюдаемые факты и может проверить их вручную.

5. Layer: где именно возникла проблема

Layer	Что проверяет	Типичный finding
PE mapping	Можно ли безопасно перейти от RVA к file offset, не выходят ли секции за файл/image.	pe_section_raw_oob, pe_section_virtual_overlap, pe_exception_directory_oob.
.pdata range map	Корректны ли RUNTIME_FUNCTION entries, сортировка, overlaps, duplicate begin, границы image.	invalid_ranges, rf_oob, rf_unsorted, rf_overlaps, duplicate_begin.
.xdata decoding	Корректно ли читается UNWIND_INFO: header, CountOfCodes, slots, padding, tail, chain.	unwind_parse_fail, unwind_anomalies.
Handler metadata	Куда ведет handler RVA и в какой секции находится обработчик.	handler_nonexec_section, handler_writable_section, handler_dominance.
Metadata placement	Где лежит unwind metadata: read-only data или executable/writable область.	unwind_in_exec_section, unwind_writable_section, unwind_unusual_section.
Reuse/dominance	Не слишком ли много функций используют одну metadata или один handler.	unwind_reuse_strong, handler_dominance, reuse_dominance_combo.
Dynamic validation	Произошло ли runtime-событие: handler hit, VEH/VCH/UEF, runtime table API.	dynamic event: rtl_add_function_table, unwind_handler_hit, veh_handler_hit.

Layer нужен, чтобы не смешивать уровни. Например, ошибка в PE mapping означает, что адресация повреждена еще до чтения .pdata. Ошибка в .xdata decoding означает, что RUNTIME_FUNCTION может быть нормальной, но metadata по UnwindInfoAddress не читается. Dynamic event означает, что что-то реально произошло при запуске, но это не отменяет static findings в неисполненных ветках.

6. Consequence: почему это важно

Consequence - это техническое последствие finding-а. Оно отвечает не на вопрос "что найдено", а на вопрос "почему это имеет значение". Без consequence finding превращается в список терминов. С consequence отчет учит: показывает, какую часть Windows x64 unwind/exception механизма может нарушить найденное наблюдение.

Например, rf_overlaps важен потому, что lookup function entry по ControlPc может стать неоднозначным. UnwindInfoRVA OOB важен потому, что Windows или инструмент не сможет надежно получить UNWIND_INFO. Writable handler target важен потому, что обработчик исключений может быть изменяемым runtime-ом или protector-логикой. Runtime function table registration важна потому, что статическая .pdata уже

не является полной картой процесса.

7. Confidence и severity: почему их нельзя путать

Confidence и severity отвечают на разные вопросы. Confidence - это уверенность в наблюдении или интерпретации. Severity - это серьезность наблюдения, если оно реально. Они могут совпадать, но не обязаны.

Confidence высокий, severity низкий: анализатор точно нашел маленький gap между функциями, но gap похож на padding и не влияет на handler route. Confidence низкий, severity потенциально высокий: есть признаки handler-driven control flow, но без dynamic validation нельзя уверенно сказать, что этот route реально используется. Confidence высокий, severity высокий: RUNTIME_FUNCTION ranges пересекаются в корректно прочитанной Exception Directory, создавая реальную неоднозначность range map.

Ситуация	Confidence	Severity	Почему
Два RUNTIME_FUNCTION диапазона пересекаются, оба корректно прочитаны из Exception Directory.	Высокий	Высокая	Наблюдение надежное, последствие серьезное: неоднозначный lookup по ControlPc.
Один unusual section name для UNWIND_INFO, но секция read-only и похожа на toolchain-specific layout.	Средний	Средняя или низкая	Факт placement есть, но benign explanation вероятен.
Handler RVA ведет в writable section и dynamic подтвердил handler hit.	Высокий	Высокая	Есть static evidence и runtime evidence.
Есть import RtlAddFunctionTable, но dynamic не запускался.	Высокий по факту import, низкий/средний по интерпретации	Низкая или средняя	Import реально есть, но он не доказывает runtime registration.
Статически подозрительный route, но target under debugger изменяет поведение.	Низкий или средний	Потенциально высокая	Риск серьезный, но evidence неполное.

8. Как это устроено в текущем проекте

В текущем коде проекта finding представлен структурой HeuristicFinding. У него есть code, family, severity, message, has_location, representative_rva и affected_count. Это значит, что текущий finding уже не является просто строкой: у него есть код правила, семейство evidence, серьезность, краткое сообщение, примерная локация и количество затронутых объектов.

Confidence сейчас хранится не внутри каждого finding-а, а на уровне HeuristicReport. Это важно для защиты: проект оценивает уверенность в общем статическом verdict-е по совокупности findings, evidence families, strong/medium counts и weakening rules. То есть confidence - это не вероятность отдельной записи, а инженерная уверенность в интерпретации набора статических сигналов.

Также в отчете есть score, suspicion, requires_manual_review, evidence_families_mask, weak/medium/strong finding counts, evidence_level и mitigations. Это делает модель reviewable: можно увидеть не только итог, но и почему confidence повышался или снижался.

```
Current HeuristicFinding fields:
  code                - stable identifier of the rule
  family              - evidence family
  severity            - seriousness class
  message             - compact human-readable message
  has_location         - whether representative RVA exists
  representative_rva   - example RVA for investigation
  affected_count       - how many records/objects are affected

Report-level fields:
  score, suspicion, confidence, requires_manual_review
  evidence_families_mask, evidence_level
  weak_finding_count, medium_finding_count, strong_finding_count
  mitigations[]
```

9. Evidence families в проекте

Family	Смысл	Какие вопросы закрывает
table_integrity	Повреждение или неоднозначность уровня PE/.pdata/RUNTIME_FUNCTION.	Можно ли доверять карте функций? Нет ли invalid ranges, overlaps, OOB, duplicate begin?
unwind_integrity	Повреждение или аномалии в UNWIND_INFO/.xdata.	Можно ли корректно прочитать header, CountOfCodes, slots, padding, tail, chain?
metadata_placement	Подозрительное размещение unwind metadata или handler target.	Почему metadata/handler лежит в executable/writable/unusual section?
unwind_reuse	Слишком сильное повторное использование одного UnwindInfoRVA.	Это нормальная дедупликация или централизованная/обфусцированная metadata?
handler_dominance	Один handler обслуживает слишком много функций.	Это известный language runtime helper или suspicious dispatcher?

Family нужна не только для красоты. Она влияет на confidence: чем больше независимых семейств evidence, тем сильнее общий static case. Один изолированный сигнал легче объяснить benign причиной. Несколько сигналов из разных слоев создают более убедительную цепочку.

10. Как score и confidence работают в проекте

Score - это инженерная оценка подозрительности в диапазоне 0..100. Он растет от findings с разными весами. Например, повреждение table_integrity или unwind_integrity обычно весит сильнее, чем одиночный unusual placement. Score отвечает на вопрос: насколько набор признаков выглядит подозрительным или существенным.

Confidence - это уверенность в интерпретации static verdict. Он не является статистической вероятностью и не должен называться "точностью модели". В проекте confidence растет от числа evidence families, strong findings, medium findings, hard table-integrity и hard unwind-integrity issues. Затем применяются weakening rules: single_signal_only, small_sample_size, placement_only_signal, single_family_only.

На защите важно сказать: это не машинное обучение и не вероятностная модель, обученная на датасете. Это явная инженерная матрица правил, которую можно читать, критиковать, менять и тестировать. Ее достоинство - прозрачность.

Понятие	Что означает	Что нельзя говорить
Score	Накопленная подозрительность по весам findings.	Нельзя говорить, что score - вероятность вредоносности.
Suspicion	Нормализованный score / 100.	Нельзя выдавать за ML probability.
Confidence	Уверенность в интерпретации static verdict.	Нельзя путать с severity и accuracy.
Severity	Серьезность отдельного finding-а.	Нельзя считать, что high severity всегда high confidence.
Mitigation	Правило, снижающее confidence из-за слабого контекста.	Нельзя считать mitigation оправданием, что проблемы нет.

11. Пример finding-a: rf_overlaps

Ниже пример того, как должен выглядеть профессиональный finding по пересечению RUNTIME_FUNCTION. Он длиннее, чем одна строка, но именно поэтому полезен: показывает evidence, layer, rule, consequence, confidence, severity и next step.

> Finding: пересечение RUNTIME_FUNCTION диапазонов.
> Evidence: RUNTIME_FUNCTION[A] покрывает RVA 0x1400..0x1480, RUNTIME_FUNCTION[B] покрывает RVA 0x1460..0x1500. Пересечение находится в диапазоне RVA 0x1460..0x1480.
> Layer: .pdata range map.
> Rule: для одного ControlPc внутри image должна существовать однозначная RUNTIME_FUNCTION entry.
> Consequence: RtlLookupFunctionEntry ищет entry по PC value; пересечение может привести к выбору не той unwind metadata или указывает на поврежденную/нестандартную таблицу.
> Confidence: высокий, если обе записи получены из корректно mapped Exception Directory и section map не содержит ошибки адресации.
> Severity: высокая, потому что нарушается однозначность карты функций.
> Next step: сверить диапазоны с disassembly в Ghidra, проверить границы функций, thunks/padding и при dynamic run посмотреть, какой active function table используется для адресов внутри пересечения.

Такой finding защищается очень хорошо. Если комиссия спрашивает "почему это проблема?", ответ уже внутри: потому что Windows unwinder должен выбрать metadata по ControlPc, а пересечение делает карту неоднозначной. Если комиссия спрашивает "почему вы уверены?", ответ: потому что диапазоны прочитаны из Exception Directory и RVA корректно сопоставлены через PE section map.

12. Пример finding-a: unwind_parse_fail

> Finding: UNWIND_INFO не удалось разобрать.
> Evidence: RUNTIME_FUNCTION[N] указывает UnwindInfoAddress = RVA 0x4020. RVA мапится в section .xdata, но CountOfCodes требует 8 slots, а до конца доступной области хватает только на 5 slots. Parser остановил разбор на поле CountOfCodes / UNWIND_CODE array.
> Layer: .xdata decoding.
> Rule: UNWIND_INFO должен содержать header, CountOfCodes slots, padding и tail в пределах доступных bytes.
> Consequence: дальнейшее чтение handler RVA или CHAININFO было бы чтением мусора. Metadata для этой функции недостоверна.
> Confidence: высокий по факту обрыва, если mapping корректен и available size посчитан по section/file bounds.
> Severity: medium или high в зависимости от доли таких записей и роли функции.
> Next step: открыть .xdata bytes в hex view, проверить CountOfCodes, padding, section bounds и возможный packer/обфускацию.

13. Пример finding-a: handler_writable_section

Finding: handler target расположен в writable section.
Evidence: UNWIND_INFO содержит UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, handler RVA = 0x56010. RVA попадает в section .xw с execute и write permissions.
Layer: handler metadata / metadata placement.
Rule: exception handler обычно должен указывать на исполняемый код, но writable handler target требует объяснения, потому что обработчик может быть изменяемым runtime-ом.
Consequence: static-интерпретация handler route становится менее надежной. Это может быть legitimate JIT/protector/runtime behavior или suspicious exception dispatcher.
Confidence: высокий по факту placement, если section permissions прочитаны из PE headers корректно. Confidence по вредоносной интерпретации ниже без dynamic validation.
Severity: medium/high в зависимости от сочетания с handler dominance, reuse, executable memory и dynamic handler hit.
Next step: дизассемблировать handler target, проверить xrefs, known language handler signatures, dynamic hits и memory protection at runtime.

14. Пример dynamic finding/event: runtime function table

Dynamic event не всегда называется finding в коде, но по смыслу он может стать частью finding-a или correlated case. Например, событие rtl_add_function_table показывает, что процесс добавил RUNTIME_FUNCTION table во время выполнения. Это не видно из static .pdata как готовая таблица.

Dynamic evidence: процесс вызвал RtlAddFunctionTable.
Evidence: event type = rtl_add_function_table, arg0 = FunctionTable pointer, arg1 = EntryCount, arg2 = BaseAddress, thread id сохранен в dynamic event.
Layer: dynamic validation / runtime function table registration.
Consequence: статическая .pdata PE-файла не является полной unwind-картой процесса в данной execution session.
Confidence: высокий по факту API hit, если breakpoint-инструментация complete и сессия не timed out до события.
Severity: сама регистрация не обязательно опасна. Severity растет, если рядом есть suspicious static placement, generated executable memory, handler hits или exception route.
Next step: snapshot runtime table через ReadProcessMemory, сопоставить BaseAddress/EntryCount/ranges и проверить execution inside registered range.

15. Что должен содержать идеальный finding в будущем отчете

Текущий проект уже хранит структурированный finding, но идеальный аналитический отчет может быть еще богаче. Ниже схема, к которой стоит стремиться в развитии проекта. Она не обязательно должна вся попасть в C-структуру HeuristicFinding; часть может формироваться на уровне JSON/HTML report.

```
Ideal finding schema:
  code: stable machine-readable rule id
  title_ru: short Russian title
  family: evidence family
  layer: PE mapping | .pdata | .xdata | handler | dynamic
  severity: low | medium | high | critical
  confidence: finding-level confidence or evidence confidence
  affected_count: number of affected objects
  representative_location:
    rva, file_offset, section, permissions
  evidence:
    concrete ranges, fields, bytes, counters, related function index
  rule:
    what invariant was checked
  expected:
    normal structure expected by PE/x64 unwind model
  observed:
    what analyzer observed
  consequence:
    why it matters technically
  alternative_explanations:
    compiler/runtime/padding/JIT/protector possibilities
  dynamic_status:
    not_run | not_observed | confirmed | contradicted | incomplete
  next_steps:
    manual validation steps for Ghidra/debugger/hex view
```

16. Как finding проходит pipeline проекта

Этап	Что происходит
1. PE parse	Файл читается как bytes. Проверяются PE headers, sections, Exception Directory RVA/Size и mapping.
2. RUNTIME_FUNCTION read	Exception Directory читается как массив entries. Фиксируются BeginAddress, EndAddress, UnwindInfoAddress.
3. UNWIND_INFO parse	Для каждой записи читаются Version, Flags, CountOfCodes, UWOP slots, padding, tail, CHAININFO/handler.
4. Summary per function	Собираются fixed_stack_bytes, saved registers, handler_rva, chain status, anomalies.
5. Heuristics	Подсчитываются признаки по families, создаются HeuristicFinding entries.
6. Score/confidence	По матрице весов считается score, suspicion, confidence, evidence_level, mitigations.
7. Dynamic optional	На Windows analyzer запускает target, собирает runtime events и correlation signals.
8. JSON/HTML	Findings дополняются rule/expected/impact/evidence chain/manual steps и становятся читаемым отчетом.

17. Как защищать finding перед комиссией

Комиссия может атаковать finding с разных сторон: "почему это считается проблемой", "почему вы уверены", "почему это не нормальный compiler output", "почему severity такая", "где доказательства". Хорошая защита строится по одной стабильной схеме.

```
Answer template:
1. ████████ layer: ███ ████████ ███████ .pdata range map / .xdata decoding / handler metadata /
dynamic validation.
2. ████████ evidence: ██████████ RVA, fields, section permissions, affected count, event type.
3. ████████ rule: ██████ ████████████ ████████ ████████████.
4. ████████ consequence: ███ ██████ ██████████ ███ ███████ ███████ ██████████ ████████████
██.
5. ████████ severity ■ confidence.
6. ████████ benign alternatives: compiler/runtime/padding/JIT/protector.
7. ████████ next step: Ghidra, hex view, debugger, dynamic run, runtime snapshot.
```

18. Готовые ответы комиссии

Вопрос комиссии	Ответ
Что такое finding в вашем проекте?	Это структурированный сигнал анализа: код правила, семейство evidence, severity, сообщение, representative RVA и affected count. В отчете он расширяется объяснением rule, expected, impact, evidence chain и manual validation steps.
Finding - это уже доказательство вредоносности?	Нет. Finding доказывает или фиксирует конкретное структурное наблюдение. Вредоносность - более высокий вывод, требующий контекста, dynamic validation и ручной проверки.
Чем severity отличается от confidence?	Severity - насколько серьезно наблюдение, если оно реально. Confidence - насколько надежно установлено наблюдение или интерпретация. Их нельзя смешивать.
Почему confidence не является точностью модели?	Потому что проект не ML-модель. Confidence - инженерная уверенность в static verdict по прозрачной матрице правил, families, strong/medium findings и weakening rules.
Почему finding длинный?	Потому что задача отчета - не только предупредить, но и показать доказательную цепочку. Реверсер должен видеть адреса, правило, последствие и следующий шаг.
Если finding может иметь benign объяснение, зачем он нужен?	Потому что reverse engineering работает с неполными доказательствами. Finding фиксирует область, которую надо проверить. Benign explanations отражаются через mitigations и reduced confidence.
Почему rf_overlaps high severity?	Потому что пересечение RUNTIME_FUNCTION ranges делает lookup по ControlPc неоднозначным. Это затрагивает основу x64 unwind range map.
Почему одиночный import RtlAddFunctionTable не high severity?	Потому что import - только static evidence подготовки. Он не доказывает runtime registration и может быть нормальным для JIT/runtime. Dynamic должен подтвердить actual call.

19. Как объяснять ошибки и ограничения

Очень важно не защищать проект неправдой. Если finding не содержит disassembly-based verification, нужно так и говорить: текущий static проверяет metadata structure и placement, но не всегда сверяет UWOP с реальным prolog. Если dynamic не запускался, нужно говорить: runtime confirmation отсутствует. Если dynamic запускался, но timed out или instrumentation incomplete, отсутствие события нельзя считать сильным доказательством.

Такая честность не ослабляет защиту. Наоборот, она показывает инженерную зрелость. Хороший инструмент не обещает невозможное. Он показывает, где факт, где гипотеза, где интерпретация и где нужна ручная проверка.

20. Что улучшить в проекте именно по findings

Приоритет	Улучшение	Зачем
P1	Добавить finding-level evidence block в JSON для каждого finding.	Чтобы каждый finding имел concrete ranges, section, file offset, expected/available sizes.
P1	Явно добавить layer в finding/report.	Чтобы не смешивать PE mapping, .pdata, .xdata, handler и dynamic.
P1	Добавить finding-level confidence или evidence confidence.	Сейчас confidence общий по report. Для защиты полезно показать уверенность по конкретному finding.
P2	Добавить alternative explanations.	Чтобы отчет сразу показывал benign compiler/runtime/JIT/padding explanations.
P2	Добавить dynamic_status для каждого relevant finding.	not_run, not_observed, confirmed, incomplete. Это свяжет static и dynamic без ложных выводов.
P2	Расширить examples в HTML.	Для сложных findings вроде rf_overlaps или CountOfCodes показывать chain of reasoning.
P3	Добавить prolog-vs-unwind verification.	Чтобы consequence опиралось не только на metadata, но и на code-vs-metadata match.

21. Финальная формула для защиты

Finding в проекте - это единица доказательного анализа. Он не равен вердикту о вредоносности и не равен простой ошибке парсинга. Он связывает конкретное наблюдение с уровнем механизма, правилом, последствием, серьезностью, уверенностью и следующим шагом ручной проверки.

Самая сильная формулировка: проект не просит поверить ему на слово. Он показывает путь рассуждения: PE mapping -> .pdata range map -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler/dynamic event -> finding -> consequence -> confidence/severity -> manual validation. Именно это отличает систему анализа от простого parser-a.